

**INSTITUTO
FEDERAL**
Santa Catarina

Sintaxe do JavaScript pt 2

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas

Revisão

- Interpretada pelo navegador
- **var / let / const** declaram variáveis
- não tipada, mas reconhece
 - números
 - textos
 - booleanos
 - funções (*aula de hoje*)
 - DOM (*semana que vem*)
- múltiplos valores ordenados -> arrays
 - não precisam ser do mesmo tipo
 - acesso pelo seu índice (ordem)

exemplos

```
const PI = 3.1415;
var nome = "Alice";
let a = [];
a[0] = true;
a[1] = PI;
a[2] = nome;
console.log(a) // ???
let b = [,...a,];
b.push("zero");
b.unshift(2.4);
console.log(b); // ???
console.log(b.length); // ???
```



Condicionais

- executam instruções condicionalmente

exemplos

Condicionais

- **executam instruções condicionalmente**
- **if** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se for verdadeira**
 - **segue** a execução do programa **se for falsa**

exemplos



Condicionais

- **executam instruções condicionalmente**
- **if** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se for verdadeira**
 - **segue** a execução do programa **se for falsa**

exemplos

```
if (nomeUsuario == null)
    nomeUsuario = "Zé Pequeno";
```

Condicionais

- **executam instruções condicionalmente**
- **if** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se** for **verdadeira**
 - **segue** a execução do programa **se** for **falsa**

exemplos

```
if (nomeUsuario == null)
    nomeUsuario = "Zé Pequeno";
```

```
if (!endereco) {
    endereco = "";
    mensagem = "Informe o endereço!";
}
```

Condicionais

- executam instruções condicionalmente
- **if-else** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se** for **verdadeira**
 - **executa outro** conjunto de instruções **se** for **falsa**

exemplos

Condicionais

- **executam instruções condicionalmente**
- **if-else** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se** for **verdadeira**
 - **executa outro** conjunto de instruções **se** for **falsa**

exemplos

```
if (n === 1)
    alert("Você tem 1 mensagem!");
else
    alert(`Você tem ${n} mensagens!`);
```



Condicionais

- executam instruções condicionalmente
- **if-else** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se** for **verdadeira**
 - **executa outro** conjunto de instruções **se** for **falsa**

exemplos

```
i = j = 1;  
k = 2;  
if (i === j)  
    if (j === k)  
        alert("i é igual a k");  
else  
    alert("i não é igual a j");
```

Condicionais

- executam instruções condicionalmente
- **if-else** permite a seleção entre dois caminhos distintos para execução
- testa uma expressão lógica
 - **executa** um conjunto de instruções **se** for **verdadeira**
 - **executa outro** conjunto de instruções **se** for **falsa**

exemplos

```
i = j = 1;
k = 2;
if (i === j) {
    if (j === k) {
        alert("i é igual a k");
    }
} else {
    alert("i não é igual a j");
}
```

Condicionais

- executam instruções condicionalmente
- **switch** permite a seleção entre dois ou mais caminhos distintos para execução
- o valor de uma expressão é verificado em uma lista de constantes
- quando a ocorrência é encontrada o bloco de instruções associado é executado
- melhor legibilidade que `if-else`

exemplos



Condicionais

- executam instruções condicionalmente
- **switch** permite a seleção entre dois ou mais caminhos distintos para execução
- o valor de uma expressão é verificado em uma lista de constantes
- quando a ocorrência é encontrada o bloco de instruções associado é executado
- melhor legibilidade que if-else

exemplos

```
switch (mes) {  
    case 1:  
        alert("janeiro");  
        break;  
    case 2:  
        alert("fevereiro");  
        break;  
    default: // Opcional  
        alert("Mês inválido");  
        break;  
}
```

Laços

- executam instruções repetidamente

exemplos

Laços

- executam instruções repetidamente
- **while** permite a repetição da execução de um bloco de instruções
- testa uma expressão lógica
 - **repete** a execução do bloco **enquanto** a expressão for **verdadeira**
 - **segue** a execução do programa **quando** a expressão for **falsa**

exemplos



Laços

- executam instruções repetidamente
- **while** permite a repetição da execução de um bloco de instruções
- testa uma expressão lógica
 - **repete** a execução do bloco **enquanto** a expressão for **verdadeira**
 - **segue** a execução do programa **quando** a expressão for **falsa**

exemplos

```
let contador = 0;  
while (count < 10) {  
    console.log(contador);  
    contador++;  
}
```

Laços

- **executam instruções repetidamente**
- **do-while** permite a repetição da execução de um bloco de instruções
- o bloco de instruções será executado pelo menos uma vez

exemplos



Laços

- executam instruções repetidamente
- **do-while** permite a repetição da execução de um bloco de instruções
- o bloco de instruções será executado pelo menos uma vez

exemplos

```
let len = a.length, i = 0;  
if (len === 0) {  
  console.log("Array vazio");  
} else {  
  do {  
    console.log(a[i]);  
  } while (++i < len);  
}
```



Laços

- **executam instruções repetidamente**
- **for** permite a repetição finita da execução de um bloco de instruções
- o início e o fim da repetição são conhecidos
- quando a contagem termina, segue a execução do programa

exemplos



Laços

- **executam instruções repetidamente**
- **for** permite a repetição finita da execução de um bloco de instruções
- o início e o fim da repetição são conhecidos
- quando a contagem termina, segue a execução do programa

exemplos

```
for (let count = 0; count < 10; count++) {  
    console.log(count);  
}
```



Laços

- executam instruções repetidamente
- **for-of** permite a repetição da execução de um bloco de instruções para cada elemento de uma coleção

exemplos



Laços

- executam instruções repetidamente
- **for-of** permite a repetição da execução de um bloco de instruções para cada elemento de uma coleção

exemplos

```
let dados = [1, 2, 3, 4, 5, 6, 7, 8, 9];  
let soma = 0;  
for (let elemento of dados) {  
    soma += elemento;  
}  
sum; // 45
```



Laços

- executam instruções repetidamente
- **for-of** permite a repetição da execução de um bloco de instruções para cada elemento de uma coleção
- funciona com qualquer objeto iterável

exemplos

```
let dados = [1, 2, 3, 4, 5, 6, 7, 8, 9];
let soma = 0;
for (let elemento of dados) {
    soma += elemento;
}
sum; // 45
```

```
let palavra = "programa";
for (let letra of palavra) {
    console.log(letra);
}
```



Laços

- executam instruções repetidamente
- **for-of** permite a repetição da execução de um bloco de instruções para cada elemento de uma coleção
- funciona com qualquer objeto iterável
- não é possível interromper a iteração com `break`

exemplos

```
let dados = [1, 2, 3, 4, 5, 6, 7, 8, 9];
let soma = 0;
for (let elemento of dados) {
    soma += elemento;
}
sum; // 45
```

```
let palavra = "programa";
for (let letra of palavra) {
    console.log(letra);
}
```



Funções

o que são

- **blocos de código definidos uma vez, mas podem ser executados inúmeras vezes**
- **parametrizadas** incluem parâmetros que funcionam como variáveis locais para a função
- usam **argumentos** para calcular um valor de **retorno**
- **são objetos em js** atribuídas a variáveis, argumentos de outras funções, etc.

exemplos



Funções

o que são

- **blocos de código definidos uma vez, mas podem ser executados inúmeras vezes**
- **parametrizadas** incluem parâmetros que funcionam como variáveis locais para a função
- usam **argumentos** para calcular um valor de **retorno**
- **são objetos em js** atribuídas a variáveis, argumentos de outras funções, etc.

exemplos

```
function somaNumeros(a, b) {  
    return a + b;  
}
```



Funções

definindo funções

- **declaração de função**

exemplos

```
function somaNumeros(a, b) {  
    return a + b;  
}
```

Funções

definindo funções

■ declaração de função

- function
- identificador (nome da função)
- parênteses com zero ou mais identificadores (parâmetros) separados por vírgula
- chaves com zero ou mais instruções JS

exemplos

```
function somaNumeros(a, b) {  
    return a + b;  
}
```



Funções

definindo funções

■ declaração de função

- function
- identificador (nome da função)
- parênteses com zero ou mais identificadores (parâmetros) separados por vírgula
- chaves com zero ou mais instruções JS
- "içadas" (hoisted) para o início do script
- sempre retornam algo

exemplos

```
function somaNumeros(a, b) {  
    return a + b;  
}
```



Funções

definindo funções

- declaração de função
- **expressão de função**
 - faz parte de uma expressão maior
 - nome opcional

exemplos

```
const quadrado = function(x){return x*x;}
```



Funções

definindo funções

- declaração de função
- **expressão de função**
 - faz parte de uma expressão maior
 - nome opcional

exemplos

```
const quadrado = function(x){return x*x;}
```

```
const f = function fatorial(x) {  
  if (x <= 1) return 1;  
  else return x*fatorial(x-1);  
};
```



Funções

definindo funções

- declaração de função
- **expressão de função**
 - faz parte de uma expressão maior
 - nome opcional
 - podem ser argumentos de outras funções (ou métodos)

exemplos

```
const quadrado = function(x){return x*x;}
```

```
const f = function fatorial(x) {  
  if (x <= 1) return 1;  
  else return x*fatorial(x-1);  
};
```



Funções

definindo funções

- declaração de função
- expressão de função
 - faz parte de uma expressão maior
 - nome opcional
 - podem ser argumentos de outras funções (ou métodos)

exemplos

```
const quadrado = function(x){return x*x;}
```

```
const f = function fatorial(x) {  
  if (x <= 1) return 1;  
  else return x*fatorial(x-1);  
};
```

```
[3,2,1].sort(function(a,b){return a-b;});
```



Funções

definindo funções

- declaração de função
- **expressão de função**
 - faz parte de uma expressão maior
 - nome opcional
 - podem ser argumentos de outras funções (ou métodos)
 - podem ser definidas e invocadas imediatamente

exemplos

```
const quadrado = function(x){return x*x;}
```

```
const f = function fatorial(x) {  
  if (x <= 1) return 1;  
  else return x*fatorial(x-1);  
};
```

```
[3,2,1].sort(function(a,b){return a-b;});
```



Funções

definindo funções

- declaração de função
- expressão de função
 - faz parte de uma expressão maior
 - nome opcional
 - podem ser argumentos de outras funções (ou métodos)
 - podem ser definidas e invocadas imediatamente

exemplos

```
const quadrado = function(x){return x*x;}
```

```
const f = function fatorial(x) {  
  if (x <= 1) return 1;  
  else return x*fatorial(x-1);  
};
```

```
[3,2,1].sort(function(a,b){return a-b;});
```

```
let f = (function(x){return x*x;})(10);
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**

exemplos

```
const sum = (x, y) => { return x + y; };
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função

exemplos

```
const sum = (x, y) => { return x + y; };
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função

exemplos

```
const sum = (x, y) => { return x + y; };
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função
 - pode ser bem compacta

exemplos

```
const sum = (x, y) => { return x + y; };
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função
 - pode ser bem compacta

exemplos

```
const sum = (x, y) => { return x + y; };
```

```
const sum = (x, y) => x + y;
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função
 - pode ser bem compacta

exemplos

```
const sum = (x, y) => { return x + y; };
```

```
const sum = (x, y) => x + y;
```

```
const quadrado = x => x * x;
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função
 - pode ser bem compacta

exemplos

```
const sum = (x, y) => { return x + y; };
```

```
const sum = (x, y) => x + y;
```

```
const quadrado = x => x * x;
```

```
const funcaoConst = () => 32;
```



Funções

definindo funções

- declaração de função
- expressão de função
- **função seta**
 - seta separa os parâmetros do corpo da função
 - funciona como uma expressão de função
 - pode ser bem compacta
 - não deve ter quebra de linha entre os parâmetros e a seta

exemplos

```
const sum = (x, y) => { return x + y; };
```

```
const sum = (x, y) => x + y;
```

```
const quadrado = x => x * x;
```

```
const funcaoConst = () => 32;
```



Funções

definindo funções

- declaração de função
- expressão de função
- função seta
- **função aninhada**
 - definida dentro de outra função

exemplos



Funções

definindo funções

- declaração de função
- expressão de função
- função seta
- **função aninhada**
 - definida dentro de outra função

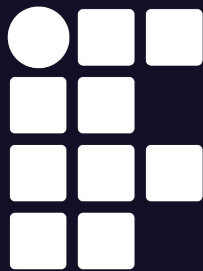
exemplos

```
function hipotenusa(a, b) {  
  function quad(x) { return x * x; }  
  return Math.sqrt(quad(a) + quad(b));  
}
```



Exercícios

Resolva a lista de exercícios do SIGAA



**INSTITUTO
FEDERAL**
Santa Catarina

Sintaxe do JavaScript

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas